PyRaft Lab

Complete 15-Day Implementation Plan

A Raft-based fault-tolerant distributed key-value store with deterministic fault injection, automated consistency testing, experiment execution, metrics, and performance analytics.

Purpose: This document converts the supplied PyRaft Lab specification into a phase-by-phase implementation plan suitable for giving directly to a coding AI.

Core rule: Build incrementally. Implement one phase, run tests, verify acceptance criteria, fix failures, and only then proceed. Do not replace required Raft mechanisms with mocks just to make tests pass.

1. Project Goal and Architecture

PyRaft Lab has two layers: the distributed system itself and the laboratory used to inject failures and measure behavior. The supplied specification describes a Raft cluster of 3–5 nodes, replicated KV state, WAL persistence, a deterministic network simulator, clients, experiment scenarios, and analytics.

Layer A — Distributed System

• Raft consensus

• Leader election

• Log replication and commit

• KV state machine

• Persistent WAL

• Client operations and leader discovery

• Linearizable reads

Layer B — Experimentation Laboratory

• Deterministic network simulation

• Latency, jitter, loss, partitions, reordering, asymmetric links and clock skew

• YAML/JSON experiment scenarios

• Workload generation

• Metrics collection

• Result JSON and plots

• Linearizability verification

Critical Raft state

```
Persistent: current_term, voted_for, log
Volatile: commit_index, last_applied, role
Leader-only: next_index, match_index
Timers: election timeout 150–300 ms, heartbeat 50 ms
```

Critical invariants: election safety, log matching, and state-machine safety.

2. Phase 0 — Foundation (Day 1)

Objective: establish a clean Python 3.11+ repository and a minimal message bus before adding Raft.

Technology: Python, asyncio, pytest/pytest-asyncio, YAML/Pydantic, matplotlib, Typer/Click; keep dependencies manageable.

```
pyraft_lab/
■■■ src/pyraft_lab/
■   ■■■ raft/
■   ■■■ kv/
■   ■■■ network/
■   ■■■ client/
■   ■■■ lab/
■   ■■■ cli/
■■■ scenarios/
■■■ tests/
■■■ results/
■■■ docs/
■■■ pyproject.toml
■■■ README.md
■■■ LICENSE
```

Implement: repository configuration, package modules, serialization, basic in-memory network bus, and CI skeleton.

Acceptance: two nodes can send and receive arbitrary messages reliably without any fault injection.

3. Phase 1 — Raft Core (Days 2–4)

Day 2 — State and RPC: implement RaftState, node roles, LogEntry, RequestVote and AppendEntries message types/handlers, term handling, and serialization.

Day 3 — Leader Election: randomized election timeout, candidate transition, self-vote, RequestVote broadcast, majority vote counting, split-vote retry, higher-term handling, leader transition, and heartbeat scheduling.

Day 4 — Log Replication: leader appends client commands, followers validate prevLogIndex/prevLogTerm, reject conflicts, leader decrements next_index and retries, and entries commit only after majority replication.

Acceptance tests:

• 3-node cluster elects exactly one leader per term.

• Leader crash results in a new election.

• 100 commands replicate consistently across nodes.

• Conflicting follower logs are repaired.

• No committed entry is applied twice or replaced by a different entry at the same index.

4. Phase 2 — KV Store and Client (Days 5–6)

Day 5 — State Machine: apply only committed entries. Support PUT, GET, DELETE and CAS. Track last_applied.

Day 6 — Client: support leader discovery, NOT_LEADER redirects, retries with exponential backoff, and request success/failure/redirect metrics.

```
client.put(key, value)
client.get(key)
client.delete(key)
client.cas(key, expected, new)
```

Week 1 milestone: a working 3-node Raft KV store with no injected faults.

5. Phase 3 — Failure Handling (Days 7–10)

Day 7 — Failure Detection and Recovery: leader sends empty AppendEntries heartbeats; followers reset election timers; stale leaders step down when seeing higher terms. Verify leader crash recovery and old-leader rejoin.

Day 8 — Network Simulator: replace direct messaging with SimulatedNetwork. Implement per-link latency and drop rate first. Use a virtual clock and deterministic seeds so failures are reproducible.

Day 9 — Partitions: configure partition groups. Minority cannot obtain quorum; majority continues operating; healing allows stale followers to catch up.

Day 10 — Linearizable Reads: implement ReadIndex (preferred) or a leader lease. During a partition, an old leader must not serve stale reads.

Fault types required by the specification:

• Latency: 0–500 ms default range

• Jitter: configurable variance

• Packet loss: 0–50%

• Network partition

• Node crash

• Asymmetric links

• Message reordering

• Clock skew

• Slow disk/WAL delay

> **Week 2 milestone:** fault injection works, leader recovery works, partitions behave according to quorum, and linearizability is addressed.

6. Phase 4 — Experiment Laboratory (Days 11–13)

Day 11 — Experiment Runner: parse YAML, spawn N nodes and clients, configure network faults, run workloads, collect events, calculate summaries, and write JSON.

```
name: leader_crash
duration: 30s
nodes: 3
faults:
  - type: node_crash
    target: leader
    start: 10s
    duration: 5s
workload:
  type: put_get_mix
  puts_per_sec: 10
```

Day 12 — Metrics and Visualization: calculate throughput, average latency, p50/p95/p99 latency, election time, recovery time, availability, replication lag, election count, data loss, stale reads, and network fault impact. Generate latency, leadership, timeline, and replication-lag plots.

Day 13 — Linearizability Verification: record every concurrent client operation with timestamps and results, then verify whether the history is linearizable. Test under partitions and leader changes.

7. Required Experiment Scenarios

Scenario	Purpose
baseline	Normal performance with no faults
leader_crash	Failover and recovery
minority_partition	Quorum rejection
majority_partition	Majority availability and election
network_latency	Tolerance to high latency
packet_loss	Handling lossy links

churn	Repeated node restarts

Bonus only if core correctness is already stable: asymmetric links, clock skew, message reorder, disk pause, cascading failures.

8. Testing Strategy

Unit tests: election, voting, term changes, log matching, conflict repair, majority commit, ordered apply, KV operations and persistence.

Integration tests: leader crash, minority partition, majority partition, partition healing, network latency and packet loss.

Property/safety tests: no conflicting committed entries, no committed data loss across repeated leader crashes, progress when a majority is available, and linearizable client histories.

Testing rule for the coding AI: every phase must end with automated tests. Do not move forward with known failing core tests.

9. Final Project Structure

```
pyraft_lab/
■■■ src/pyraft_lab/
■   ■■■ raft/
■   ■   ■■■ node.py
■   ■   ■■■ state.py
■   ■   ■■■ rpc.py
■   ■   ■■■ log.py
■   ■   ■■■ election.py
■   ■   ■■■ replication.py
■   ■   ■■■ persistence.py
■   ■■■ kv/
■   ■   ■■■ statemachine.py
■   ■   ■■■ operations.py
■   ■■■ network/
■   ■   ■■■ simulator.py
■   ■   ■■■ link.py
■   ■   ■■■ clock.py
■   ■■■ client/
■   ■   ■■■ client.py
■   ■   ■■■ workload.py
■   ■■■ lab/
■   ■   ■■■ runner.py
■   ■   ■■■ scenario.py
■   ■   ■■■ metrics.py
■   ■   ■■■ visualization.py
■   ■■■ cli/
■       ■■■ main.py
■■■ scenarios/
■■■ tests/
■■■ results/
■■■ docs/
■■■ pyproject.toml
■■■ README.md
■■■ LICENSE
```

10. Final CLI

```
pyraft-lab start
pyraft-lab put key value
pyraft-lab get key
pyraft-lab status
pyraft-lab show-log
pyraft-lab run --scenario baseline
pyraft-lab run --scenario leader_crash
pyraft-lab run --scenario minority_partition
pyraft-lab results
```

11. Final Metrics and Result Format

Each experiment should record scenario name, run ID, duration, total requests, successful requests, redirects/failures, availability, data loss, average/p50/p95/p99 commit latency, election count/time, leader timeline, messages sent/delivered/dropped/delayed, and recovery/replication statistics.

```
results/{scenario}/
■■■ results.json
■■■ timeline.png
■■■ latency.png
■■■ leadership.png
■■■ replication_lag.png
```

12. Risk Controls

Risk	Mitigation
Election loops	Randomized timeouts and split-vote handling
Log divergence	Strict prevLogIndex/prevLogTerm checks and conflict truncation
Stale reads	ReadIndex or leader lease
Persistence corruption	WAL integrity checks and safe persistence before acknowledgement
Async race conditions	Single-threaded per-node event handling; locks only where needed

Flaky tests	Deterministic simulator, seeded randomness, virtual clock

13. Day 14 — Documentation and Benchmarking

• README with installation and quickstart.

• Architecture diagram.

• Technical report covering design decisions, fault tolerance, results and lessons.

• Run the final 7-scenario benchmark suite with repeated trials.

• Save reproducible result files and plots.

14. Day 15 — Demo and Release

• Run full integration suite.

• Fix final blocking bugs.

• Record the 5-minute demonstration.

• Show baseline → leader crash → partition → recovery → analytics → linearizability.

• Tag v0.1.0 and publish the release.

> **Demo target:** 3 nodes boot; 100 writes replicate; leader crashes; new leader is elected; no data loss; minority partition rejects writes; majority continues; high latency/loss slows but does not break the system; analytics show latency and leadership timelines; linearizability check passes.

15. Master Instruction to Give the Coding AI

Use the following as the controlling instruction:

Build PyRaft Lab incrementally over the 15-day implementation plan. Follow the phase order exactly: foundation → Raft state/RPC → election → replication → KV → client → failure recovery → deterministic network → partitions → linearizable reads → experiment runner → metrics/visualization → linearizability checker → documentation/benchmarks → final release. For every phase, first implement the required files and real mechanisms, then run the relevant tests, inspect failures, fix them, and report the phase acceptance criteria. Do not use mock Raft logic, hard-coded leaders, fake fault results, or hard-coded metrics. Keep all components modular and testable. Preserve deterministic behavior using seeded randomness and a virtual clock. Do not add unnecessary technologies or features that threaten the 15-day scope. Never silently skip a required feature; if a feature cannot fit, explicitly identify it and preserve the core Raft correctness first.

16. Definition of Done

• 3–5 node Raft cluster works correctly.

• Leader election and failover are real and tested.

• Log replication and majority commit work.

• KV state machine supports PUT/GET/DELETE/CAS.

• Client discovers and follows the leader.

• WAL/persistence is implemented as specified.

• Deterministic network simulator supports required fault types.

• Seven minimum scenarios run from YAML.

• Metrics and plots are generated automatically.

• Linearizability is checked from recorded client histories.

• 40+ meaningful automated tests are targeted.

• README, architecture documentation, benchmark results and demo are included.

Source Basis

This plan is derived from the supplied **PyRaft Lab — The Definitive 15-Day Build** specification, including its architecture, Raft state model, fault types, scenarios, experiment runner, metrics, 15-day schedule, project structure, testing strategy, risks, and demo requirements.

■filecite■turn0file0■L468-L564■